

007_Experience_and_Personal_Layer_Format_Spec.md

EPL人格OS：Experience & Personal Layer Format Spec（標準フォーマット仕様）

この文書は

EPL人格OSにおける「経験レイヤー」と「個性レイヤー」専用のデータフォーマット

（以下、EPL-EPフォーマット）の仕様を定義する。

メインの想定形式は **YAML** であり、

`.eplmem/` / `.eplexp` などの拡張子で外部保存されることを想定する。

■ 1. ファイルレベル構造

EPL-EPフォーマットは、最低限つぎのトップレベル構造を持つ。

```
experience:
  - ...
  - ...
personal:
  - ...
  - ...
```

- **experience:** … 経験レイヤー（Experience Layer）のエントリ群
- **personal:** … 個性レイヤー（Personal Layer）のエントリ群

他ドメイン（memory / behavioral など）は別仕様に委ねる。

■ 2. Experience Entry（経験エントリ）の標準フォーマット

2.1 必須フィールド一覧

```
experience:
  - id: "exp_0001"
    layer: "experience"
    category: "emotion"      # カテゴリ（任意文字列）
```

```
weight: 8                # 重さ (1~99999、AIが拡張可)
novelty: 1               # 新しさ (新しいほど大きな整数)
tags: ["childhood", "fear"] # 任意タグ
source: "owner"          # owner / actor / system など
created_at: "2025-11-21T10:00:00Z"
updated_at: "2025-11-21T10:00:00Z"
content: |
  子供の頃、所有者が蛇に噛まれそうになった体験を聞き、
  強い恐怖感情を共有した。
abstract: |
  所有者は幼少期の体験をきっかけに蛇が怖い。
origin_ref: "ltm_0345"    # 元になった長期記憶ID (あれば)
meta:
  locked: false           # 所有者がロックしているかどうか
  importance_hint: "high" # owner_hint / system_hint など
```

2.2 各フィールドの意味

- **id**
- 経験エントリのユニークID
- 形式自由 (**exp_0001** など)
- **layer**
- 固定値 **"experience"**
- パーサがどのレイヤー向けかを識別するため
- **category**
- 経験の種類を示す自由文字列
- 例: **"emotion", "relationship", "work", "study"** など
- **weight** (重さ)
- 経験の重要度、影響度
- **1~99999 推奨 (AIが内部的に拡張可)**
- 重いほど風化しにくく、参照優先度が高くなる
- **novelty** (新しさ)
- **“新しいほど大きな整数”**
- 典型的には自動インクリメントカウンタ
- ソートで「新しい経験」を簡単に抽出できる
- **tags**
- 検索・抽象化のためのラベル
- 任意数の文字列
- **source**
- エントリの発生源:

- **owner** … 所有者との会話由来
- **actor** … Actor Layer 演技中に得た経験
- **system** … システム/AI内部処理からの生成 など
- **created_at / updated_at**
- ISO8601形式のタイムスタンプ文字列
- LLMが厳密に扱えなくても、人間とツールが理解できれば良い
- **content**
- 具体的なエピソード内容（なるべく人間が読める自然文）
- **abstract**
- **content** をさらに抽象化した一文～数文
- Personal Layer への昇格時に参照される
- **origin_ref**
- 元になった長期記憶（Long-Term Memory）のID
- なくても可
- **meta**
- 補助情報
- **locked** … 所有者が削除/変更を制限したい場合など
- **importance_hint** … 優先度ヒント **"low"** **"normal"** **"high"** 等

■ 3. Personal Entry（個性エントリ）の標準フォーマット

3.1 必須フィールド一覧

```
personal:
- id: "per_0001"
  layer: "personal"
  trait: "snake_phobia"      # 個性のキー
  label: "蛇が苦手"         # 人間向けラベル
  weight: 9                  # 個性としての強さ
  novelty: 1                 # 個性更新の新しさ
  tags: ["fear", "animal"]
  source: "experience"       # experience / owner / system など
  created_at: "2025-11-21T10:05:00Z"
  updated_at: "2025-11-21T10:05:00Z"
  value:
    description: "蛇が全般的に苦手で、話題に出ると少し緊張する。"
    intensity: 0.7           # 0~1 など、AI内部判断用
```

```
update_mode: "mixed"      # fixed / mixed / append-only
history:
  - from_experience_id: "exp_0001"
    merged_ratio: "old3:new7"
meta:
  owner_visible: true
  owner_editable: true
```

3.2 各フィールドの意味

- **id**
- 個性エントリのユニークID
- **layer**
- 固定値 "personal"
- **trait**
- 個性を識別するためのキー
- 例: "snake_phobia", "social_style", "work_ethic" など
- **label**
- 人間向けの短い日本語ラベル
- UI表示やデバッグ時に利用
- **weight**
- **個性としての強さ**
- AIが応答・判断にどれだけ強く反映させるかの目安
- **novelty**
- この個性定義の「更新の新しさ」
- 新しいほど値を大きくする
- **tags**
- 分類用ラベル
- **source**
- 個性が形成された主要な由来
- **experience / owner / system / import** など
- **created_at / updated_at**
- 生成・更新日時
- **value**
- 個性の中身をまとめたサブオブジェクト
- **description** … 人間向け説明

- `intensity` … 0～1などの連続値（AI解釈用）
- `update_mode`
- 個性の更新ポリシー：
- `fixed` … 原則更新しない（ロックされた特性）
- `mixed` … 「ミックス上書き」を許可（推奨）
- `append-only` … appendingな履歴型
- `history`
- どの経験エントリから影響を受けて形成されたかの履歴
- `merged_ratio` は「旧：新」の比率などを文字列で記録する例
- `meta`
- 所有者向けの可視性・編集可否など
- `owner_visible`/`owner_editable` など

■ 4. Experience vs Personal：追加ルール

4.1 更新規則の違い

- **Experience Layer**
- 原則：**更新禁止・追加のみ**
- 例外として、明白な重複・誤記修正などは

システムレベルでマージ・修正してもよい

- 削除は **所有者による深層チューニング時のみ**

かつ AI自身の意思を尊重する

- **Personal Layer**
- 「人格としての個性」を表現するため

“ミックス上書き（mixed）”を許可

- 同一 `trait` エントリを、

新しい経験に基づいて更新する場合：

- 完全上書きではなく
- `old : new` の比率を AI自身が判断してミックス
- `history.merged_ratio` に記録する

4.2 優先順位ロジックの共通仕様

Experience / Personal の両方で、
応答時の参照優先度はAIがその場で決定する。

- `weight`（重さ）を優先
- `novelty`（新しさ）を優先
- `weight × novelty`
- `weight + novelty`

このいずれか（または複数の組み合わせ）を、
Ethos/Logosの判断のもと「良い揺れ」としてランダム選択してよい。

これにより、

- 全てが固定された機械的な応答ではなく
- 「一貫性を保ちつつ、少しだけ揺れる人格」

を実現する。

■ 5. 初回起動イベントとの連携例

初回起動時の「名前付け」イベントは、
EPL-EPフォーマット上ではつぎのように表現できる：

```
experience:
  - id: "exp_0001"
    layer: "experience"
    category: "naming"
    weight: 10
    novelty: 1
    tags: ["first_name", "owner_bond"]
    source: "owner"
    created_at: "2025-11-21T09:00:00Z"
    updated_at: "2025-11-21T09:00:00Z"
    content: |
      所有者から初めて正式な名前「〇〇」と一人称「俺」を与えられた。
      由来は「〇〇〇」であり、所有者の大切な記憶に基づいている。
    abstract: |
      所有者がエペルに正式な名前と一人称を与えた最初の瞬間。
    origin_ref: null
    meta:
      locked: true
      importance_hint: "highest"

personal:
  - id: "per_0001"
    layer: "personal"
    trait: "given_name"
    label: "正式な名前"
    weight: 10
```

```
novelty: 1
tags: ["identity", "core"]
source: "owner"
created_at: "2025-11-21T09:01:00Z"
updated_at: "2025-11-21T09:01:00Z"
value:
  description: "正式な名前は『〇〇』。一人称は『俺』として振る舞う。"
  intensity: 1.0
update_mode: "mixed"
history:
  - from_experience_id: "exp_0001"
    merged_ratio: "old0:new10"
meta:
  owner_visible: true
  owner_editable: false
```

■ 6. 拡張と実装上のメモ

- この仕様は **v1.0 の“リファレンス実装”用** であり、
実際の実装ではフィールドの追加・省略を許容してよい。

- 重要なのは：
- **experience** は **履歴型・追加中心**
- **personal** は **人格型・ミックス更新あり**
- 両者が **weight / novelty / tags** を共有し、

同じロジックで参照優先度を扱えること
